

VNUHCM - University of Science
Faculty of Information Technology

CS486 – Lecture 4

Query Languages

Le Thi Nhan

ltnhan@fit.hcmus.edu.vn

Content

- Introduction
- Define relation schemas
- Manipulate on relation schemas
 - Data modification
 - **Modification operations**
- Summary

Content

- Introduction
- Relational algebra
- Set operations
- Selection
- Projection
- Cartesian product
- Join operation
- Division operation
- Other operations
- **Update operations**

Update operations

- The content of the database can be updated by update operations
 - Insertion
 - Deletion
 - Update
- These operations are expressed by an assignment operation

$$\mathbf{R}_{new} \leftarrow \text{operations on } \mathbf{R}_{old}$$

Insertion operation

- Is expressed

$$R_{new} \leftarrow R_{old} \cup E$$

- R is a relation
- E is a relational algebra expression

- Example

- Assign the employee whose SSN is 123456789 to the project 20, with the number of working hours is 10

$WORKS_ON \leftarrow WORKS_ON \cup ('123456789', 20, 10)$

Deletion operation

- Is expressed

$$R_{new} \leftarrow R_{old} - E$$

- R is a relation
- E is a relational algebra expression

- Example

- Delete all work assignments of the employee 123456789

$WORKS_ON \leftarrow WORKS_ON - \sigma_{SSN='123456789'}(WORKS_ON)$

Example 23

- Remove project assignments that have locations in “Houston”

Update operation

- Is expressed

$$R_{new} \leftarrow \pi_{F1, F2, \dots, Fn} (R_{old})$$

- R is a relation
- Fi is an arithmetic expression that results in the new value for attributes

- Example

- Increase working hours to 1.5 times for all employees

$$WORKS_ON \leftarrow \pi_{SSN, PNo, Hours*1.5}(WORKS_ON)$$

Example 24

- Increase working hours to 1.5 times for assignments that are over 30 hours, the remain will be increased to 2 times

Content

- Introduction
- Define relation schemas
- Manipulate on relation schemas
- **Summary**

Relational algebra operation

- Set operations
 - Union $\cup$
 - Intersection $\cap$
 - Difference $-$
- Retrieve parts of a relation
 - Selection σ
 - Projection π
- Combine tuples of two relations
 - Cartesian product $\times$
 - Join $\bowtie$
- Aggregate values
 - Grouping & aggregate functions $\mathcal{F}$

SQL statements

- Data definition
 - CREATE, ALTER, DROP...
- Data manipulation
 - SELECT
 - FROM
 - WHERE
 - GROUP BY
 - HAVING
- View definition
- Index design

Relational calculus

■ Categories

- Tuple relational calculus (TRC)
 - SQL
- Domain relational calculus (DRC)
 - QBE (Query By Example) MS Access
 - DataLog (Database Logic)

Formal definition - TRC

- A general expression is of the form

$$\{ t_1.A_i, t_2.A_j, \dots, t_n.A_m \mid P(t_1, t_2, \dots, t_n, \dots, t_{n+m}) \}$$

- $t_1, t_2, \dots, t_n$ are tuple variables
- $A_i, A_j, \dots, A_m$ are attributes of tuples t
- P is a condition or well-formed formula
 - P is made up of predicate calculus atoms

Tuple variable

■ Free variable

$$\{ t \mid t \in \text{EMPLOYEE} \wedge t.\text{SALARY} > 30000 \}$$

 t is a free variable

■ Bound variable

$$\{ t \mid t \in \text{EMPLOYEE} \wedge \exists s \in \text{DEPARTMENT} (s.\text{DNUMBER} = t.\text{PNO}) \}$$

 Free variable

 Bound variable

Atoms

- (i) $t \in R$

$t \in \text{EMPLOYEE}$

 - t is a tuple variable
 - R is a relation

- (ii) $t.A \theta s.B$

$t.\text{SSN} = s.\text{ESSN}$

 - A is an attribute of the tuple variable t
 - B is an attribute of the tuple variable s
 - θ is comparison operators, eg. $<, >, \leq, \geq, \neq, =$

- (iii) $t.A \theta c$

$t.\text{SALARY} > 30000$

 - C is a constant
 - A is an attribute of the tuple variable t
 - θ is comparison operators, eg. $<, >, \leq, \geq, \neq, =$

Atoms

- Each of atoms evaluates to either TRUE or FALSE for a specific combination of tuples
- Formula (i)
 - TRUE value if t is a tuple of the specified relation R
 - FALSE value if t does not belong to R

R	A	B	C
	α	10	1
	α	20	1

$t1 = \langle \alpha, 10, 1 \rangle$

$t2 = \langle \alpha, 20, 2 \rangle$

$t1 \in R$ has the TRUE value

$t2 \in R$ has the FALSE value

Atoms

- Formula (ii) and (iii)
 - If the tuple variables are assigned to tuples such that they satisfy the condition, then the atom is TRUE

R	A	B	C
	α	10	1
	α	20	1

If t is the tuple $\langle \alpha, 10, 1 \rangle$

Then $t.B > 5$ has the TRUE value ($10 > 5$)

Rules

- (1) Every atom is formula
- (2) If P is a formula then
 - $\neg P$ is a formula
 - (P) is a formula
- (3) If P_1 and P_2 are formulas then
 - $P_1 \vee P_2$ is a formula
 - $P_1 \wedge P_2$ is a formula
 - $P_1 \Rightarrow P_2$ is a formula

Rules

- (4) If $P(t)$ is a formula then
 - $\forall t \in R (P(t))$ is a formula
 - TRUE when $P(t)$ is TRUE for all tuples in R
 - FALSE when there is one tuple that makes $P(t)$ FALSE
 - $\exists t \in R (P(t))$ is a formula
 - TRUE when there exists some tuple that makes $P(t)$ TRUE
 - FALSE when $P(t)$ is FALSE for all tuples t in R

Rules

- (5) If P is an atom then
 - Tuple variables t in P are free variables

- (6) Formulas $P = P_1 \wedge P_2$, $P = P_1 \vee P_2$, $P = P_1 \Rightarrow P_2$
 - A variable t in P is free or bound variable will depends on its role in P_1 and P_2

Transform

- (i) $P_1 \wedge P_2 = \neg (\neg P_1 \vee \neg P_2)$
- (ii) $\forall t \in R (P(t)) = \neg \exists t \in R (\neg P(t))$
- (iii) $\exists t \in R (P(t)) = \neg \forall t \in R (\neg P(t))$
- (iv) $P \Rightarrow Q = \neg P \vee Q$

Safe expression - TRC

- Examine

$$\{ t \mid \neg(t \in \textit{EMPLOYEE}) \}$$

- Unsafe

- Many tuples in the universe that are not EMPLOYEE tuples
- Even though they do not exist in the database
- The result is infinitely numerous

Safe expression - TRC

- Safe expression
 - Guarantee to yield *a finite number of tuples*

- A formula P is called safe expression
 - If its resulting values are from the domain of P
 - The domain of a tuple relational calculus expression: $DOM(P)$
 - The set of all values
 - * Either appear as constant values in P
 - * Or exist in any tuple in the relation referenced in P

Safe expression - TRC

■ Example

$$\{ t \mid t \in \text{EMPLOYEE} \wedge t.\text{SALARY} > 30000 \}$$

- $\text{DOM}(t \in \text{EMPLOYEE} \wedge t.\text{SALARY} > 30000)$
- The set of values
 - Larger than 30000 at the attribute SALARY
 - Other values at the remaining attributes that appear in EMPLOYEE
- Safe expression

Safe expression - DRC

■ Examine

$$\{ p, r, s \mid \neg (\langle p, q, r, s, t, u, v, x, y, z \rangle \in \text{EMPLOYEE}) \}$$

- Values in the result do not belong to the domain of the expression
- Unsafe

Safe expression - DRC

■ Examine

$$\{ x \mid \underbrace{\exists y (<x, y> \in R)}_{\text{Formula 1}} \wedge \underbrace{\exists z (\neg <x, z> \in R \wedge P(x, z))}_{\text{Formula 2}} \}$$

- R is a relation with a finite number of values
- We also have a finite number of values that does not belong to R
- Formula 1: examine values in R only
- Formula 2: could not validate cause we do not know the finite number of values of variable z

Safe expression - DRC

■ Expression

$$\{ x_1, x_2, \dots, x_n \mid P(x_1, x_2, \dots, x_n) \}$$

is safe if :

- Values that appear in tuples of the expression must belong to the domain of P
- $\exists$ quantifiers: expression $\exists x (Q(x))$ is TRUE iff
 - Values of x belong to $DOM(Q)$ and make $Q(x)$ TRUE
- $\forall$ quantifiers: expression $\forall x (Q(x))$ is TRUE iff
 - $Q(x)$ is TRUE for all values of x belonging to $DOM(Q)$

Content

- View
- Index
- CASE structure
- WITH statement
- Suquery in Select/From clause

View

- Table is a relation that exist actually the database
 - Stored in some physical organization
 - Persistent
- View is also a relation
 - Do not exist physically (virtual table)
 - Do not contain the data
 - Is derived from other tables
 - Can query or even modify the data through views

Why use views

- Hide the complexity of data
- Simplify queries
- Present data with the purpose “easy to use”
- Mechanism of data safety

Definition

■ Syntax

```
CREATE VIEW <View_name> AS  
    <Query>
```

```
DROP VIEW <View_name>
```

■ View contains

- A list of attributes that are the same as attributes in SELECT clause
- The number of tuples depending on the conditions in WHERE clause
- Data derived from tables in FROM clause

Example

```
CREATE VIEW EMP_DEP5 AS
```

```
    SELECT SSN, LName, Minit, FName
```

```
    FROM   EMPLOYEE
```

```
    WHERE DNo=5
```

```
CREATE VIEW STATISTIC_DEP AS
```

```
    SELECT DNo, DName, COUNT(*) AS Number_Emp,  
           SUM(Salary) AS Total_Salary
```

```
    FROM EMPLOYEE, DEPARTMENT
```

```
    WHERE DNo=DNumber
```

```
    GROUP BY DNumber, DName
```

Querying views

- Although views do not contain data, we can do the query on views

```
SELECT FName  
FROM EMP_DEP5  
WHERE LName LIKE 'Smith'
```

$$\text{EMP_DEP5} \leftarrow \pi_{\text{SSN, LName, Minit, FName}} (\sigma_{\text{DNo}=5} (\text{EMPLOYEE}))$$
$$\pi_{\text{FName}} (\sigma_{\text{LName}='Smith'} (\text{EMP_DEP5}))$$

Querying views

- Can query data from both tables and views

```
SELECT LName, FName, PName, Hours  
FROM EMP_DEP5, WORKS_ON, PROJECT  
WHERE SSN=ESSN AND PNo=PNumber
```

$$\text{EMP_DEP5} \leftarrow \pi_{\text{SSN, LName, Minit, FName}} (\sigma_{\text{DNo}=5} (\text{EMPLOYEE}))$$
$$\text{TMP} \leftarrow \text{EMP_DEP5} \bowtie_{\text{SSN=ESSN}} (\text{WORKS_ON} \bowtie_{\text{PNo=PNumber}} \text{PROJECT})$$
$$\pi_{\text{LName, FName, PName, Hours}} (\text{TMP})$$

Modifying views

- Can apply INSERT, DELETE, and UPDATE commands to simple views
 - Views built on one table and having the key attribute of that table

- Cannot modify views
 - Views have a key word DISTINCT
 - Views use aggregate functions
 - Views have extended SELECT clause
 - Views are derived from table containing constraints on columns
 - Views are derived from many tables

Modifying views

- Modify the last name of employee 123456789 in department 5 to “Ford”

```
UPDATE EMP_DEP5  
SET      LName= 'Ford'  
WHERE SSN= '123456789'
```

Index design

- The index on an attribute A is the data structure that makes it efficient to find tuples having a fixed value for the attribute A

```
SELECT *  
FROM   EMPLOYEE  
WHERE  DNo=5 AND Sex= 'Female'
```

Read 10.000 tuples

Read 200 tuples

Table EMPLOYEE has 10.000 tuples

There are 200 employees who work for the department 5

Read 70 tuples

Index

■ Syntax

```
CREATE INDEX <Index_name> ON <Table_name>(<Column_name>)
```

```
DROP INDEX <Index_name>
```

■ Example

```
CREATE INDEX DNo_IND ON EMPLOYEE(DNo)
```

```
CREATE INDEX DNo_Sex_IND ON EMPLOYEE(DNo, Sex)
```

Discussion

- Speed up queries in which a value for an attribute is specified, join operations
- Make insertion, deletion, and update more complex and time-consuming
- Cost
 - Index storage
 - Disk access (HDD vs. SSD)
- Selection of indexes
 - One of the principal factors that influence a database
 - One of the hardest parts of database design

Example

- Examine the relation WORKS_ON(ESSN, PNo, Hours)
- Assume that
 - WORKS_ON is stored in 10 disk blocks
 - The cost of examining the entire relation WORKS_ON is 10
 - On the average, an employee works on 3 projects and a project has 3 employees
 - Tuples for a given employee or project are likely to be spread over the 10 disk blocks
 - The cost of finding 3 tuples for an employee or a project will take 3 disk accesses (if we have indexes)
 - Using indexes to locate tuples
 - The cost of reading an index's block is 1 disk access
 - Insertion costs 2 disk accesses

Example

- Suppose that the following queries are performed frequently

- Q1 `SELECT PNo, Hours`
 `FROM    WORKS_ON`
 `WHERE ESSN= '123456789'`
- Q2 `SELECT ESSN`
 `FROM    WORKS_ON`
 `WHERE PNo=1 AND Hours=20.5`
- Q3 `INSERT INTO WORKS_ON`
 `VALUES ( '123456789' , 1, 20.5)`

Example

- The costs of each of the 3 queries

Queries	No index	ESSN index	PNo index	Both indexes
Q1	10	4	10	4
Q2	10	10	4	4
Q3	2	4	4	6
Average cost	$2 + 8p_1 + 8p_2$	$4 + 6p_2$	$4 + 6p_1$	$6 - 2p_1 - 2p_2$

The fraction of the time we do Q1 is p_1

The fraction of the time we do Q2 is p_2

The fraction of the time we do Q3 is $1 - p_1 - p_2$

Usage

- Difference between
 - Scan an entire table to find tuples matching a condition
 - Locate tuples immediately using an index
- Basic data structures
 - Balanced tree (=, <, >, ≤, ≥, ...)
 - Hash table (=)

Example

```
SELECT ESSN  
FROM   WORKS_ON  
WHERE  PNo=1 AND Hours>20.5
```

- Index on PNo
 - Quickly find tuples having $PNo=1$, then check to see if $Hours>20.5$
- Index on Hours
 - Quickly find tuples having $Hours>20.5$, then check to see if $DNo=1$
- Index on (PNo, Hours)
 - Simultaneously find tuples having $PNo=1$ and $Hours>20.5$

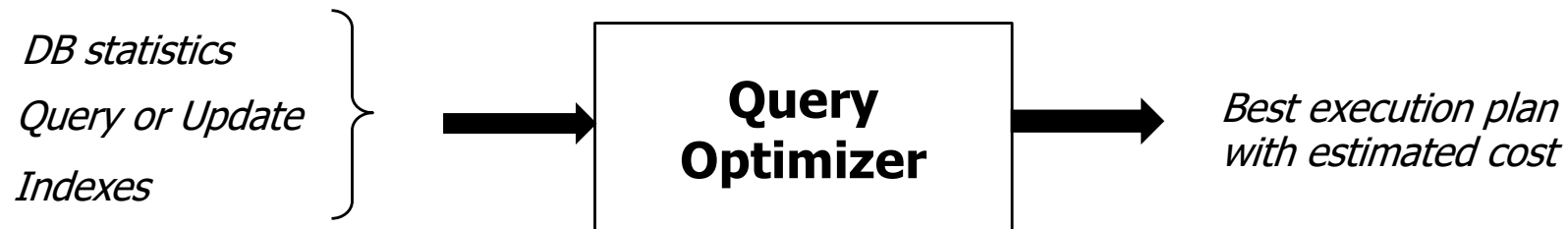
Example

```
SELECT LName, FName, DName  
FROM   EMPLOYEE, DEPARTMENT  
WHERE  DNo=DNumber
```

- Index on DNumber
- Index on DNo
- Index on (DNumber, DNo)
 - If Employee and Department relations are in sorted order of indexed attributes
 - Then a merge-like operation of the two indexes can be used to match employee and department tuples

Discussion

- Which indexes to build on a database ?
 - Database
 - How large the tables are & their data distribution
 - Workload
 - Queries and updates that are expected to be performed & their frequency



- Experiments with different setups of indexes
 - Take a look at queries and updates that we expect
 - How often we expect them to happen
 - Which indexes will benefit these queries
 - Overhead will not incur when there are updates

CASE structure

- Allows us to check conditions or output the information in each case
- Syntax

```
CASE <Column_name>  
    WHEN <Value> THEN <Expression>  
    WHEN <Value> THEN <Expression>  
    ...  
    [ELSE <Expression>]  
END
```

Example

- Retrieve the last and first name of employees who are gonna reach the retired age (male: 60 years old, female: 55 years old)

```
SELECT LName, FName
FROM   EMPLOYEE
WHERE  YEAR(GETDATE()) - YEAR(BirthDate) >=
      (CASE Sex
        WHEN 'Male' THEN 60
        WHEN 'Female' THEN 55
      END)
```

Example

- Retrieve the last and first name of employees, as well as their retired year

```
SELECT LName, FName,  
       (CASE Sex  
         WHEN 'Male' THEN YEAR(BirthDate) + 60  
         WHEN 'Female' THEN YEAR(BirthDate) + 55  
       END ) AS RetiredYear  
FROM EMPLOYEE
```

WITH statement

- Allows us to define *temporary relations*, recursive or not (SQL-99)
- Syntax

WITH

[Recursive] R1 **AS** (Definition of R1),

[Recursive] R2 **AS** (Definition of R2),

...

[Recursive] Rn **AS** (Definition of Rn)

Query involving R1, R2, ..., Rn

Recursion

■ Syntax

WITH RECURSIVE R

(

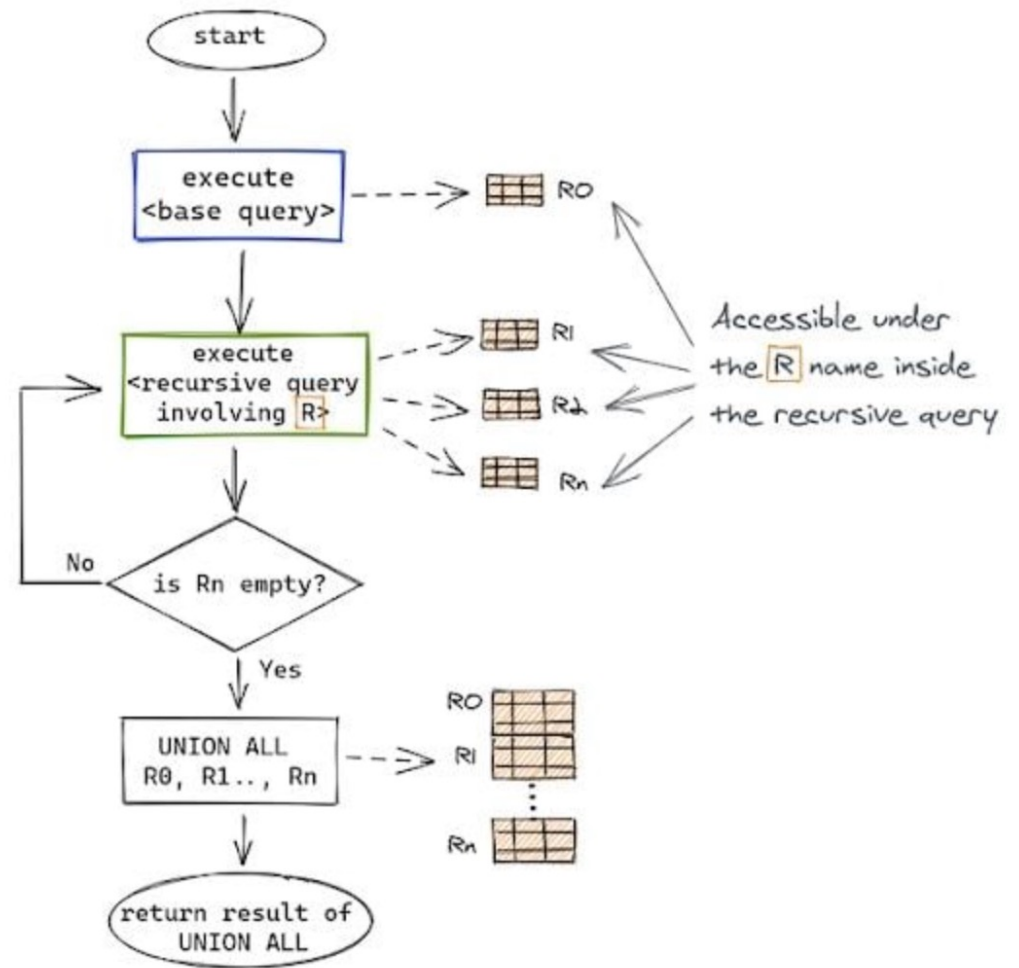
<Base query>

UNION [ALL]

<Recursive query involving R>

)

Query involving R



<https://builtin.com/data-science/recursive-sql>

Example

- Retrieve employees of a supervisor at all levels

```
WITH SUP_EMP(SupSSN, EmpSSN) AS
(
    SELECT SuperSSN, SSN
    FROM EMPLOYEE
    UNION ALL
    SELECT e.SuperSSN, e.SSN
    FROM EMPLOYEE AS e, SUP_EMP AS s
    WHERE e.SuperSSN=s.EmpSSN
)
SELECT *
FROM SUP_EMP
```

Example

- Retrieve a department with more than 5 employees and its number of employees whose salary are larger than 40,000

```
WITH BIG_DEPTS (Dno) AS
(
    SELECT Dno
    FROM EMPLOYEE
    GROUP BY Dno
    HAVING COUNT (*) > 5
)
SELECT e.Dno, COUNT (*)
FROM EMPLOYEE e, BIG_DEPTS d
WHERE e.Salary>40000 AND e.Dno=d.Dno
GROUP BY e.Dno
```

Example

- Retrieve a department with more than 5 employees and its number of employees whose salary are larger than 40,000

```
WITH BIG_DEPTS (Dno) AS (  
    SELECT Dno  
    FROM EMPLOYEE  
    GROUP BY Dno  
    HAVING COUNT (*) > 5  
)  
SELECT e.Dno, COUNT (*)  
FROM EMPLOYEE e, BIG_DEPTS d  
WHERE e.Salary>40000  
AND e.Dno=d.Dno  
GROUP BY e.Dno
```

```
SELECT e.Dno, COUNT (*)  
FROM EMPLOYEE e  
WHERE e.Salary>40000  
AND e.Dno IN (  
    SELECT Dno  
    FROM EMPLOYEE  
    GROUP BY Dno  
    HAVING COUNT (*) > 5 )  
GROUP BY e.Dno
```

Subquery in From

```
CREATE VIEW EMP_DEP5 AS  
SELECT SSN, LName, Minit, FName  
FROM EMPLOYEE  
WHERE DNo=5
```

```
SELECT LName, FName, PName, Hours  
FROM EMP_DEP5, WORKS_ON, PROJECT  
WHERE SSN=ESSN AND PNo=PNumber
```

should not

```
SELECT LName, FName, PName, Hours  
FROM WORKS_ON, PROJECT,  
    ( SELECT SSN, LName, Minit, FName  
      FROM EMPLOYEE  
      WHERE DNo=5) AS EMP_DEP5  
WHERE SSN=ESSN AND PNo=PNumber
```

Subquery in Select

product (maker, model, type)

PC (model, speed, ram, hd, price)

laptop (model, speed, ram, hd, screen, price)

printer (model, color, type, price)

- Find the average price of PC's and laptops made by manufacturer "D".
- Find the average price of computers made by manufacturer "D".

Next week

- Integrity Constraint
- Read
 - Chapter 7 [The complete book, 2E]

